UNIVERSITÉ LIBRE DE BRUXELLES
Faculté des Sciences — Département d'Informatique

INFO-F-404 - Real-Time Operating Systems

Project

Joël Goossens Attilio Discepoli

1 Project objectives

In the scope of this project, you must consider both uniprocessor and multiprocessor systems and periodic tasks. You will be required to implement different main parts:

- A Rate Monotonic (RM) scheduler.
- A Deadline Monotonic (DM) scheduler.
- An Earliest Deadline First (EDF) scheduler.
- A partitioner (for multiprocessor systems).

Note : *Although the multiprocessor section hasn't yet been covered in class, you can start the project with the uniprocessor part (i.e. setting the number of processors to 1 ($m = 1$)).*

2 Evaluation

The project will be defended during an oral presentation at the end of the semester. You have to be able to explains the code as well as the logic behind the scheduling algorithms, feasibility interval, etc.

3 Project details

This project should be implemented individually. There are two deliverables: the code itself (written in the programming language of your choice[1]) and a PDF report. Be careful to structure and document your code properly.

3.1 Task set file

A task set file is a file that contains a description of the tasks that have to be scheduled. This file has four columns separated by semicolon that represents the offset, WCET, deadline and period `(offset;wcet;deadline;period)`. An example is shown at Listing 1.

```
70;10;50;50
0;20;120;80
0;10;40;100
20;50;200;200
```

Listing 1 - Example of taskset

[1]Python, C or Rust are common choices. If you want to use another language, please get in touch by email or Teams.

MEMBRE DE L'ACADÉMIE UNIVERSITAIRE WALLONIE-BRUXELLES ET DU PÔLE UNIVERSITAIRE EUROPÉEN BRUXELLES-WALLONIE

You are provided with some sample tasksets on the UV. Please note that these do not cover all possible combinations, so you will need to add more to cover every scenario (you can refer to the examples in the curriculum guide for this).

3.2 Scheduling algorithms

You are required to implement three scheduling algorithms: *Rate Monotonic* (RM), *Deadline Monotonic* (DM) and *Earliest Deadline First* (EDF). For each algorithm should implement at least the following two main features:

- **Feasibility interval**: given a task set, compute the approriate feasibility interval.
- **Priority assignement**: given a set of jobs, return the one with the highest priority.

3.3 Feasibility intervals

Because we consider any combination of task set (up to asynchronous with arbitrary deadlines) and scheduler (FTP and FJP), one of your tasks in this project is to identify the optimal (smallest) feasibility interval to consider for each combination, if any.

Many of these combinations are discussed in the course (and presented in the curriculum guide), but some are absent. For those combinations, you should display a meaningfull error message saying why this combination is not supported and terminate early. You may construct a table with the feasibility interval according to the combination (if any).

3.4 Scheduler simulators

The scheduler simulator is the core of this project. Its purpose is to check wheter the given task set is schedulable with the given parameters (scheduling algorithm, number of cores).

- Whenever is possible, the simulator should determine the schedulability of the system without performing the simulation.
- Otherwise, it should run the simulation for every core until the end of their respective feasibility interval or until a deadline is missed on any of the cores.

Note : *Because we consider arbitrary deadlines, you should design a scheduling policy that takes into account the possibility of multiple jobs from the same task running simultaneously.*

3.5 Partitioner

The purpose of the partitioner is to partition the task set between the different processors according to a combination of sorting order and heuristic.

You have to implement *First Fit*, *Next Fit*, *Best Fit* and *Worst Fit* heuristics and you should be able to order task by increasing or decreasing order of utilization. For instance, you might use the FF (First-Fit) and DU (Decreasing Order) combination to partition the tasks among the cores.

The partitioning of a task set can fail if $U_\pi(\tau) > 1$ for any processor π, in which case you should terminate early (cf. Section 4.2).

MEMBRE DE L'ACADÉMIE UNIVERSITAIRE WALLONIE-BRUXELLES ET DU PÔLE UNIVERSITAIRE EUROPÉEN BRUXELLES-WALLONIE

4 Running the program

4.1 Program inputs

Your program should accept multiple arguments on the command line as shown in the below example:

```
$ ./project <task_file> -a edf|rm|dm [-m <#cores>] [-h ff|bf|nf|wf] [-s du|iu]
```

-a The `-a` parameter (for algorithm) is mandatory and can take one of the values in `edf|rm|dm`.

-m The `-m` parameter is optional and can take any strictly positive integer and indicate the number of identical cores. The default value is 1.

-h The `-h` parameter (for heuristic) is optional and can take one of the values in `ff|bf|nf|wf`, which respectivly stands fot *First-Fit*, *Best-Fit*, *Next-Fit* and *Worst-Fit*. The default value is `ff`.

-s The `-s` parameter (for sort) is optional and can take one of the values in `du|iu` which stands for *decreasing utilization* and *increasing utilization*. The default value is `du`.

4.2 Program outputs

At the end of the execution, the **exit code** of the program should be:

- **0**, if the system is schedulable,
- **1**, if the partitioning fails,
- **2**, if the task model/algorithm combination is not supported,
- **3**, if the system is not schedulable (determined without simulation),
- **4**, if the system is not schedulable (determined through simulation)

The code should be **deterministic** (i.e the code must terminate and not create infinite loops).

5 Report

You have to write a report that contains at least:

- a short description about the project (max 3 pages),
- a short user guide that explains how to run the program,
- if applicable, what you have done in addition to the requirements,
- anything you think is relevant to justify your implementation choices.

6 Evaluation

This project will be graded out of 20 points and aims to evaluate your **understanding of the course theory**. As we are unable to check that only by looking at the code; an **oral defense** of ~15 minutes per student will be organised at the end of the semester.

Remark: The use of AI, while not recommended, is not prohibited. However, it is essential for students to thoroughly understand their code and the theory behind its implementation, since the oral defense will assess this understanding.

7 Handing in your project

You have to hand a zip file containing at least:

- your source code (not compiled, without useless files (.git*, .vscode, etc.))
- your report in PDF.

You have to upload this zip file on the « Université Virtuelle » no later than **8 december 2026, 23h59**.

8 Bonus: Visualization tool

Note : *This part is optional and is aimed to help you visualizing the execution of a task set.*

You can implement a visualization tool that presents a graphical representation of the execution such as the example in Fig. 1. Deadline misses should be clearly indicated and job releases can be indicated.

You are free to use any graphical libraries to plot the graph.

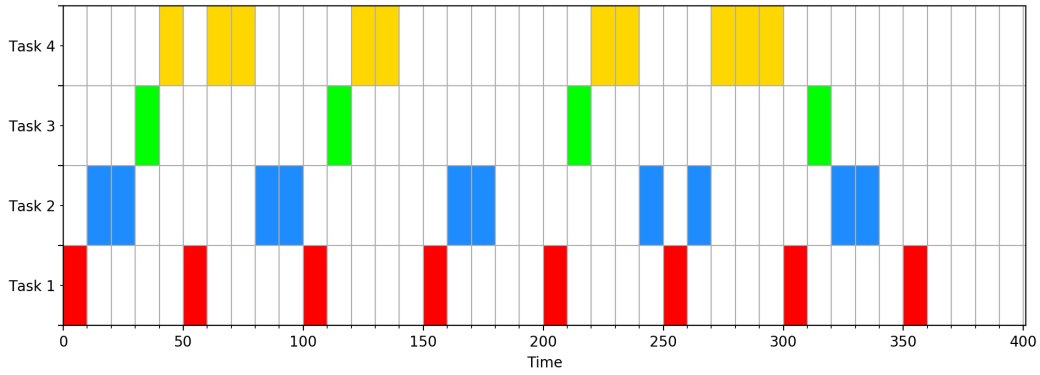


Fig. 1 - Scheduling on uniprocessor graphical representation example

MEMBRE DE L'ACADÉMIE UNIVERSITAIRE WALLONIE-BRUXELLES ET DU PÔLE UNIVERSITAIRE EUROPÉEN BRUXELLES-WALLONIE